ATIVIDADE ACADÊMICA · ENGENHARIA DE SOFTWARE

Manual de Gestão de Código e Entrega de Software

Sistema DKRT: Auditor de Conformidade de Entidades de Contexto

Branches, ambientes, revisão, CI/CD, hotfix, feature flags e organização das aplicações

Grupo: [preencher com os nomes dos integrantes]

Repositório: TCC-CEFSA-DKRT/TCC_DKRT (GitHub)

Versão 1.1 · outubro de 2026

Sumário

Apresentação e contexto..... 3
1. Estratégia de versionamento..... 5
2. Processo de desenvolvimento..... 8
3. Integração Contínua (CI)..... 10
4. Entrega e implantação (CD)..... 12
5. Tratamento de bugs em produção..... 14
6. Funcionalidades ainda não concluídas..... 15
7. Arquitetura da solução e pipelines..... 16
8. Decisões e justificativas..... 18
9. Estado atual e plano de adoção..... 19
Referências..... 20

Como ler: as seções 1 a 7 respondem, na ordem, às sete perguntas do enunciado. Em cada uma, a caixa **Estado atual** diz o que já existe no repositório hoje e o que é meta; a seção 9 consolida essa diferença num plano de adoção.

Figuras: Figura 1 (workflow Git, seção 1) e Figura 2 (pipeline de CI/CD e hotfix, seção 3).

Apresentação e contexto

Este manual define como o grupo gerencia o código-fonte do sistema DKRT e como cada alteração chega aos usuários. A regra de ouro é simples: **a branch `main` é a produção**. Nada entra nela sem antes passar por um ambiente de desenvolvimento (`develop`) e por um ambiente de homologação (`release/X.Y.Z`), cada um com aprovação da equipe de qualidade (Q.A), e só o Tech Lead autoriza o último passo.

O sistema em uma linha

O DKRT audita entidades de um Context Broker (Orion, padrão NGSI-v2) contra os Smart Data Models, em dois níveis: validação estrutural e heurística determinística (Nível 1) e análise semântica por um SLM local com RAG (Nível 2). O código é um monolito modular em arquitetura hexagonal (`domain`, `application`, `infrastructure`, `interfaces`), com um frontend Angular separado.

O que é entregue e onde roda

Componente	Tecnologia	Produção (ADR-0017)	Self-hosted (ADR-0019)
API + Worker	Python 3.11, FastAPI; um único pacote `src/auditor`	3 funções AWS Lambda (api, worker, worker-dlq) de uma mesma imagem de contêiner no ECR, atrás de um ALB	Um processo uvicorn (API + worker) em contêiner (`selfhosted/Dockerfile`)
Frontend	Angular 19	Build estático em S3 + CloudFront	Build estático servido por nginx (`selfhosted/frontend.Dockerfile`)
Banco, fila e RAG	PostgreSQL + pgvector; migrations Alembic; SQS	RDS e SQS	Postgres em contêiner; fila em memória
SLM	Ollama (Qwen2.5)	Instância EC2 com GPU (ADR-0013/0015)	Contêiner Ollama
Infraestrutura	Terraform (`infra/terraform`)	Ambientes `homolog` e `production`	Não se aplica
dataspace-demo	Stack didática com docker compose próprio	Não é implantada	Não se aplica; fornece os 3 cenários de teste de aceitação

Premissas do grupo

- Equipe pequena de estudantes trabalhando em paralelo no mesmo repositório (privado, organização GitHub do projeto), com três papéis: **Dev** (quem desenvolve), **Q.A** (quem testa e aprova) e **Tech Lead** (quem revisa, cria a release e autoriza produção).
- Um produto, uma linha de produção e **quatro ambientes**: local, **Dev**, **homologação** e **produção**. O modo self-hosted (ADR-0019) é um canal de distribuição, não um quinto ambiente do time.
- A qualidade é garantida em **dois ciclos de teste do Q.A**: o primeiro no Dev, o segundo na homologação, sobre a release candidata.
- Processo proporcional ao porte: rigoroso o bastante para que a produção nunca receba código não aprovado, leve o bastante para ser seguido por todos.

Resumo das decisões

Pergunta do enunciado	Decisão do grupo
1. Versionamento	Fluxo em funil, derivado do Git Flow: `main` = produção; `develop` = ambiente Dev; `release/X.Y.Z` = homologação. Branches de trabalho (`feature/`, `improve/`, `bug/`) **nascem da `main`**, e cada uma tem uma irmã `<nome>_develop` usada só para chegar à `develop`. Releases com tag `vX.Y.Z`.
2. Processo de desenvolvimento	Branch da `main` → teste local → `_develop` (merge da `develop` e conflitos resolvidos ali) → PR para `develop` → Q.A no Dev → PR para `main` enviado ao Tech Lead → release → Q.A na homologação → merge na `main`.
3. CI	A cada PR e a cada push em `develop` e `release/*`: testes do backend e do frontend, qualidade, segurança, validação de Terraform e build das imagens. Nenhum merge sem tudo verde.
4. CD	Só a `develop` implanta por branch (push). Homologação e produção implantam por **tag**: `vX.Y.Z-rc.N` publicada na release sobe a homologação; o merge na `main` seguido da tag `vX.Y.Z` e da **aprovação manual do Tech Lead** sobe a produção, promovendo o mesmo artefato testado.
5. Bug em produção	Branch `bug/` criada a partir da `main`, com **atalho**: sem a etapa `develop`, direto para release de correção (PATCH), homologação focada e produção. Depois, a `main` volta para a `develop`.
6. Funcionalidade inacabada	A branch de trabalho **é** a unidade de entrega: só vai para release quando pronta e aprovada. Feature flag (`FEATURE_*`) é a exceção, para o que precisa entrar em produção em partes.
7. Arquitetura	Monorepo com um só fluxo de branches, **pipelines independentes por pasta** e componentes implantáveis separadamente (frontend, API/worker e infraestrutura).

1. Estratégia de versionamento

1.1 Qual workflow Git foi adotado

O grupo adota um **fluxo em funil**, inspirado no Git Flow, com uma diferença importante. No Git Flow clássico as branches de trabalho nascem da `develop`. Aqui elas **nascem da `main`**, isto é, da produção. Por isso, o que chega à release contém apenas aquela alteração sobre o que já está em produção, e nada do que outros integrantes ainda estão testando na `develop`.

As branches permanentes são duas, e cada uma corresponde a um ambiente: a `main` (produção) e a `develop` (Dev). A homologação usa uma branch temporária, a `release/X.Y.Z`, criada pelo Tech Lead a partir da branch de trabalho aprovada no Dev. Não há uma branch permanente de homologação: a subida é feita por **tags** publicadas na release (`vX.Y.Z-rc.N`) e, depois do merge, na `main` (`vX.Y.Z`). Para chegar à `develop` sem contaminar a branch de trabalho, cada alteração ganha uma branch irmã com o sufixo `_develop`, onde o merge da `develop` e a resolução de conflitos acontecem.

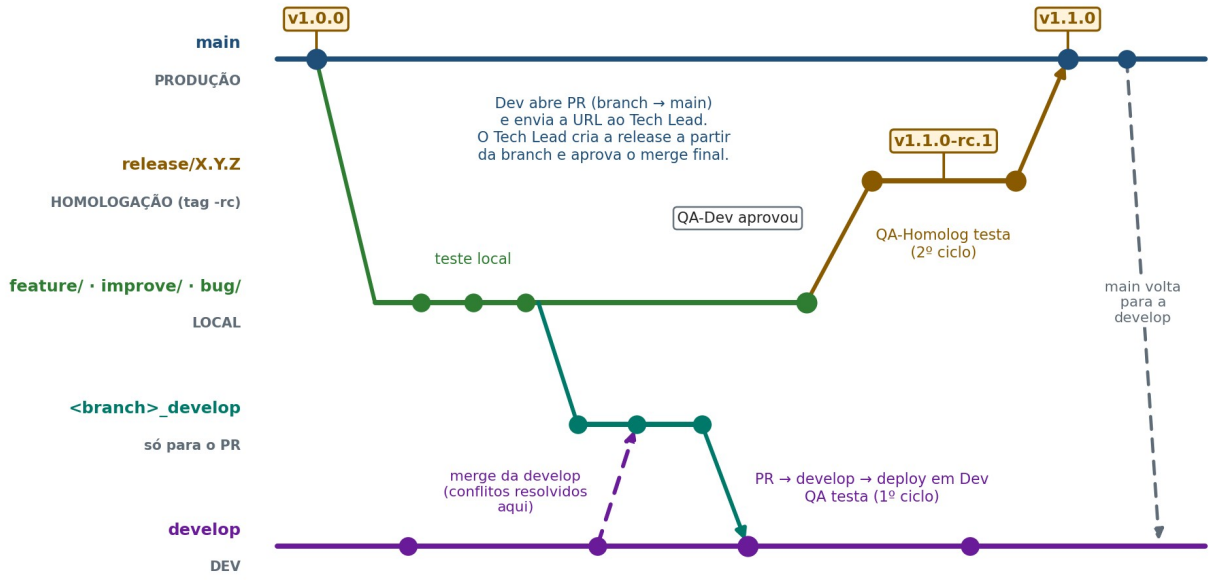


Figura 1: Workflow Git adotado. Cada linha é uma branch; o rótulo abaixo do nome indica o ambiente. As tags -rc sobem a homologação e a vX.Y.Z sobe a produção.

1.2 Branches e finalidades

Branch	Finalidade	Origem	Vida útil e encerramento
`main`	Produção. Só recebe releases aprovadas pelo Q.A na homologação. Cada release recebe uma tag.	Não se aplica	Permanente. Protegida: sem push direto, sem force push, sem exclusão. Só o Tech Lead faz merge.
`develop`	Ambiente Dev: reúne o que cada Dev está entregando para os primeiros testes do Q.A. Não é fonte de release.	`main`	Permanente. Nunca é mergeada na `main`. Recebe a `main` de volta após cada release.

Branch	Finalidade	Origem	Vida útil e encerramento
`feature/<tema>` **`improve/<tema>`** **`bug/<tema>`**	Branch de trabalho: nova funcionalidade, melhoria (inclui refatoração e documentação) ou correção de bug.	`main` atualizada	Até a release ser mergeada na `main`; então é apagada. Nunca recebe merge da `develop`.
`<branch>_develop` ex.: **`feature/login_develop`**	Veículo do PR para a `develop`. Recebe o merge da `develop`, onde os conflitos são resolvidos.	A branch de trabalho, após o teste local	Apagada assim que o PR para a `develop` é mergeado.
`release/X.Y.Z` ex.: **`release/1.2.0`**	Candidata a produção. A tag `vX.Y.Z-rc.N` publicada nela implanta na homologação; é o que o Q.A testa e o que será mergeado na `main`.	A branch de trabalho aprovada no Dev, atualizada com a `main`	Criada pelo Tech Lead. Apagada após o merge na `main`; a tag permanece.
Tags `vX.Y.Z-rc.N` e `vX.Y.Z`	É a tag que dispara a implantação. A `-rc.N`, publicada na release, sobe a homologação; a `vX.Y.Z`, publicada na `main` após o merge, sobe a produção.	Release (`-rc.N`) ou `main` (final)	Imutáveis. Nunca são movidas nem apagadas. Cada correção na homologação gera um novo `-rc`.

1.3 Regras que mantêm o fluxo coerente

O fluxo só funciona se estas cinco regras forem seguidas sempre:

1. **A branch de trabalho nunca recebe merge da `develop`.** Ela deve conter apenas a alteração do Dev sobre a `main`; é ela que vira release.
2. **Conflitos com a `develop` são resolvidos na `_develop`**, nunca na branch de trabalho e nunca no PR.
3. **Correções pedidas pelo Q.A nascem na branch de trabalho** e depois são repassadas à `_develop` (merge da branch de trabalho na `_develop`). Assim, o que foi testado no Dev e o que vai para a release são o mesmo código.
4. **A `main` só recebe releases**, e a `develop` nunca é mergeada na `main`.
5. **Depois de cada release, a `main` volta para a `develop`**, para que o Dev nunca fique atrás da produção.

1.4 Quando criar e quando encerrar uma branch

- **Criar** a branch de trabalho a partir da `main` recém-atualizada, com um objetivo pequeno e verificável, de preferência registrado numa issue. O prefixo indica o tipo: `feature/` (nova funcionalidade), `improve/` (melhoria, refatoração, documentação) e `bug/` (correção).
- **Encerrar** todas as branches da alteração (`_develop`, `release/X.Y.Z` e a de trabalho) quando a release for mergeada na `main`. O GitHub apaga a branch automaticamente após o merge de PR (opção *Automatically delete head branches*); a tag mantém o registro.
- **Branch parada:** sem commits há 7 dias, a branch é revisada em reunião do grupo: retomada, fatiada ou descartada. Como a branch de trabalho sai da `main`, ela precisa trazer a `main` para dentro dela com frequência para não acumular conflitos.
- **Mensagens de commit** seguem Conventional Commits (`feat:`, `fix:`, `docs:`, `refactor:`, `test:`, `chore:`), o que alimenta as notas de release.
- **Versionamento semântico** (`vMAJOR.MINOR.PATCH`): MAJOR para quebra de contrato (API REST ou migração incompatível), MINOR para funcionalidade ou melhoria compatível, PATCH para correção de bug. O nome da release acompanha a versão (`release/1.2.0`) e as tags de homologação ficam

`v1.2.0-rc.1`, `v1.2.0-rc.2` e assim por diante. API e frontend compartilham a mesma versão porque são liberados juntos. A primeira release será `1.0.0`, correspondente à entrega para a banca.

> **Estado atual no repositório (verificado em 05/10/2026)**
>
> • Existem `main` e branches de trabalho com nomes livres (`improve/refatoracao_simulador_data_space`, `Create-a-glossary-of-synonyms-for-RAG`). O prefixo `improve/` já é usado e continua válido; as demais passam a seguir a convenção.
>
> • Não existe nenhuma tag. O `pyproject.toml` declara a versão `0.1.0`.
>
> • O histórico mostra o problema que a regra 1 resolve: commits de merge e merges da `main` dentro de branches de trabalho (PRs #16 a #19).

2. Processo de desenvolvimento

2.1 Como uma funcionalidade é iniciada e integrada

1. **Issue.** A alteração é registrada no GitHub Issues com objetivo e critério de aceite.
2. **Branch de trabalho.** O Dev cria a branch a partir da `main` atualizada: `git switch main && git pull --ff-only && git switch -c feature/login`.
3. **Desenvolvimento e teste local.** Ciclos pequenos, com teste escrito junto do código. Antes de seguir, o Dev roda `pytest tests/unit`, `ng test` e o ambiente local (`docker compose up -d`, `uvicorn`, `ng serve`); para mudanças na auditoria, também os cenários do `dataspace-demo`.
4. **Branch `_develop`.** A partir da branch de trabalho: `git switch -c feature/login_develop`, depois `git merge origin/develop`. Os conflitos são resolvidos **aqui**, os testes locais rodam de novo e a branch sobe para o GitHub.
5. **PR para a `develop`.** O PR `feature/login_develop` → `develop` roda o CI, recebe 1 aprovação de outro integrante e é mergeado.
6. **Primeiro ciclo de testes (Dev).** O merge implanta automaticamente no ambiente Dev e o Q.A faz os primeiros testes. Se reprovar, a correção é feita na **branch de trabalho** (passo 3) e repassada à `_develop` por merge, abrindo um novo PR para a `develop`.
7. **PR para a `main`.** Com a aprovação do Q.A no Dev, o Dev abre o PR da branch de trabalho para a `main` (descrição, link da issue, evidência dos testes) e **envia a URL ao Tech Lead**.
8. **Revisão e release.** O Tech Lead revisa o código do PR. Antes de continuar, a branch de trabalho traz a `main` para dentro de si, se ela tiver andado, e o Tech Lead cria `release/X.Y.Z` **a partir dessa branch**.
9. **Segundo ciclo de testes (homologação).** O Tech Lead publica a tag `vX.Y.Z-rc.1` na release. A tag dispara o deploy na homologação, onde o Q.A testa a candidata. Se reprovar, a correção entra na branch de trabalho, o Tech Lead atualiza a release e publica `-rc.2`, repetindo o ciclo.
10. **Produção.** Com a aprovação do Q.A, o Tech Lead faz o merge da `release/X.Y.Z` na `main`, publica a tag `vX.Y.Z` (que dispara o deploy em produção) e aprova a implantação (seção 4).
11. **Encerramento.** A `main` é mergeada de volta na `develop`; as branches `_develop`, `release/X.Y.Z` e de trabalho são apagadas; a issue é fechada.

2.2 Pull Request, aprovação e verificações

Existem dois tipos de PR, com rigor proporcional ao risco do destino:

Pergunta	PR para `develop`	PR para `main` (release)
Haverá PR?	Sim, da `_develop`.	Sim, da branch de trabalho. Não há push direto na `main`.
Quem aprova?	Um integrante diferente do autor.	O **Tech Lead**, que revisa o código. O Q.A registra no PR a aprovação do Dev e, depois, da homologação.
Quem faz o merge?	O revisor ou o autor, após a aprovação.	Somente o **Tech Lead** (ou o substituto que o grupo designar), e só depois da aprovação do Q.A na homologação.

Pergunta	PR para `develop`	PR para `main` (release)
O que é verificado antes do merge?	1) CI verde (seção 3); 2) `_develop` atualizada com a `develop` e sem conflitos; 3) testes locais executados (checklist do PR).	1) CI verde na release; 2) release atualizada com a `main`; 3) Q.A aprovou no Dev **e** na homologação; 4) migrações e `dataspace-demo` verificados (seção 3.2); 5) ADR em `docs/adr/` se houver decisão arquitetural.

2.3 Por que este desenho

- **Isolar conflitos na `_develop`:** a `develop` mistura o trabalho de vários Devs. Se os conflitos fossem resolvidos na branch de trabalho, ela passaria a carregar código alheio, e a release levaria para produção algo que ninguém aprovou.
- **Dois ciclos de Q.A:** o primeiro, no Dev, é rápido e barato e detecta erro cedo; o segundo, na homologação, valida exatamente a candidata a produção, isolada das outras alterações.
- **Tech Lead como porteiro da `main`:** com vários integrantes alterando partes diferentes do sistema, um único responsável por revisar, criar a release e mergear garante que nada chegue à produção sem revisão e sem a aprovação do Q.A, e dá a quem responde pela entrega o controle do que entra.

2.4 Aplicação das regras na plataforma

As regras devem ser impostas pelo GitHub: *ruleset* na `main` e em `release/*` (somente o Tech Lead na lista de bypass; PR obrigatório; sem force push; sem exclusão) e na `develop` (PR obrigatório com CI verde). **Limite descoberto na prática:** o repositório é privado e a organização está no plano gratuito, e o próprio GitHub avisa que *rulesets* e proteção de branch **não são aplicados** nessa combinação, o que também vale para a exigência de aprovadores em *Environments*. As saídas são: upgrade da organização para o plano Team, usar benefícios acadêmicos do GitHub, ou hospedar o repositório numa conta com plano Pro. Enquanto isso não for decidido, as regras valem **por acordo do grupo**, apoiadas pelo CI e pelo CODEOWNERS.

> **Estado atual no repositório (verificado em 05/10/2026)**
>
> • PRs já são usados (#16 a #19), mas sem regras impostas: qualquer integrante com permissão de escrita consegue mergear.
>
> • Não existem `CODEOWNERS` nem template de PR (a pasta `.github` contém apenas um workflow).
>
> • Evidência do problema que o fluxo resolve: a branch `improve/refatoracao_simulador_data_space` teve o PR #17 mergeado em 23/09, e os 6 commits seguintes (27 a 28/09) ficaram de fora da `main` até o PR #19, em 05/10, exigindo resolução de conflitos.

3. Integração Contínua (CI)

3.1 O que acontece automaticamente após a submissão

O pipeline roda em **todo Pull Request** para a `develop` ou para a `main` e em todo push na `develop` e em `release/*`. Além disso, uma execução **agendada semanal** repete as verificações de segurança (novas vulnerabilidades surgem mesmo sem commits) e os testes *golden* do SLM. Cada pasta do monorepo tem seu workflow, acionado por filtro de caminho (seção 7), de modo que uma alteração só no frontend não executa os testes pesados do backend.

Etapa	O que executa	Bloqueia o merge?
Testes do backend	`pytest tests/unit` com cobertura. Em seguida, job de integração com PostgreSQL/pgvector e Orion como serviços, rodando `alembic upgrade head` em banco vazio, `alembic heads` (deve haver uma só) e `pytest -m integration`.	Sim
Testes do frontend	`npm ci`, `ng test` em Chrome headless e `ng build --configuration production`.	Sim
Qualidade de código	`ruff check` e `mypy`, adotados de forma gradual (primeiro nos arquivos alterados).	Sim, após a adoção
Segurança	`pip-audit` e `npm audit --omit=dev` (vulnerabilidades críticas e altas), varredura de segredos e Dependabot.	Sim (críticas e altas)
Infraestrutura	Quando o PR toca `infra/terraform/**`: `terraform fmt -check`, `validate` e `plan` (o `plan` é publicado para leitura).	`fmt`/`validate`: sim. `plan`: revisão humana
Build das imagens	`docker build` da imagem Lambda (`Dockerfile`) e das imagens self-hosted (`selfhosted/`), sem publicar.	Sim
Testes *golden* do SLM	`pytest -m golden` contra um Ollama real. Pesado demais para cada PR; roda no agendamento e sob demanda.	Não (alerta)

3.2 Quando uma versão está apta para implantação

Cada ambiente tem seu portão. Um código só passa para o ambiente seguinte quando **todas** as condições do portão forem verdadeiras:

Para entrar em	Condições	Quem aprova
Dev (`develop`)	CI verde no PR da `_develop`; `_develop` atualizada com a `develop`, sem conflitos; teste local feito.	1 integrante, diferente do autor
Homologação (tag `vX.Y.Z-rc.N`)	Q.A aprovou no Dev; PR para a `main` revisado; branch de trabalho atualizada com a `main`; CI verde na release.	Tech Lead (publica a tag)
Produção (`main` + tag)	Q.A aprovou na homologação; migrações Alembic aplicam em banco vazio **e** sobre o esquema da versão em produção; `GET /health` respondendo; os 3 cenários do `dataspace-demo` (um conforme, um com erro heurístico, um com erro semântico) classificados como esperado; nenhum segredo no repositório; a `main` não difere do commit homologado além do merge; notas de release geradas.	Tech Lead (aprovação manual do deploy)

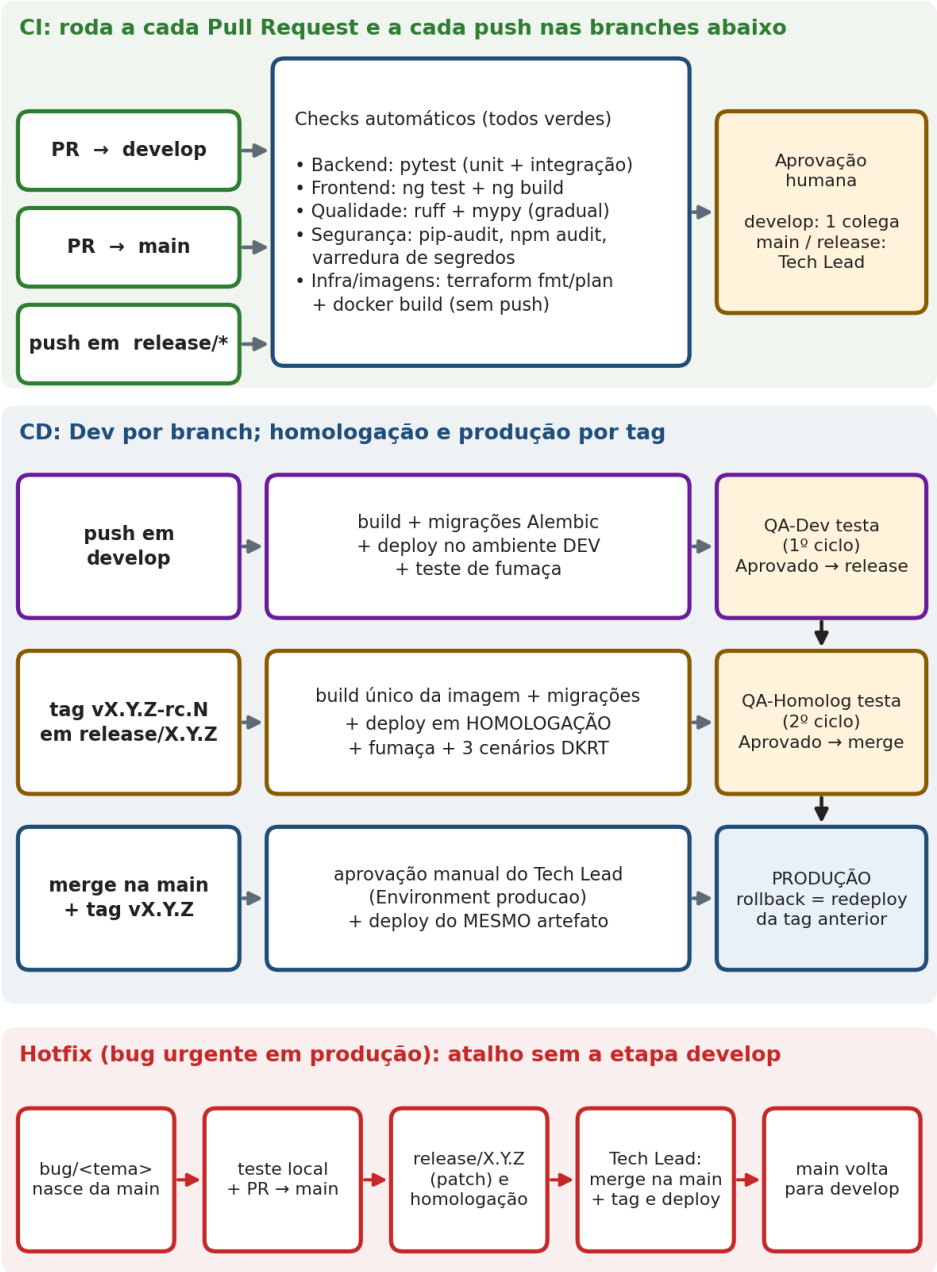


Figura 2: Pipeline de CI/CD por ambiente e fluxo de hotfix.

Estado atual no repositório (verificado em 05/10/2026)

• Existe um único workflow, `terraform-and-audit.yml`: `terraform plan` em PRs que tocam infraestrutura, `terraform apply` manual com aprovação, e `pip-audit`. O `pip-audit` só é disparado quando o PR toca `infra/` (o filtro de caminho do workflow), então não cobre PRs de código.

• O workflow versionado ainda aponta para `environments/academy`, ambiente aposentado pela ADR-0018. A versão corrigida para `homolog` já está pronta em `Claude outputs/terraform-and-audit-homolog.yml` e falta ser instalada em `.github/workflows/`.

• Os testes existem (56 arquivos em `tests/unit`, 11 em `tests/integration`, 1 em `tests/golden`, com marcadores `integration` e `golden`) e o frontend tem `ng test`, mas **nenhum deles roda em CI hoje**. Não há configuração de `ruff` nem de `mypy` (débito já registrado no próprio workflow).

4. Entrega e implantação (CD)

4.1 Como o software chega a cada ambiente

Só o **Dev** é implantado por branch: todo merge na `develop` sobe sozinho. A **homologação e a produção são implantadas por tag**. Qualquer release pode ser homologada e levada à produção, sem uma branch permanente por ambiente: o Tech Lead cria uma `release/X.Y.Z` nova para cada alteração, publica nela a tag `vX.Y.Z-rc.N` (homologação) e, depois do merge, publica a tag `vX.Y.Z` na `main` (produção). Entre as duas vale o princípio **construir uma vez, promover o mesmo artefato**: a imagem Docker e o build do Angular gerados na tag `-rc` são os que sobem para produção, sem recompilar. Para isso ser verdade, a release precisa estar atualizada com a `main` antes de ser homologada, e o pipeline de produção confere que a `main` não difere do commit homologado; se a `main` andar durante a homologação, a release é atualizada e uma nova `-rc` é publicada. O artefato do Dev nunca é promovido, porque a `develop` contém código ainda não aprovado.

Ambiente	Branch e gatilho	Automático	Aprovação humana
Local	Nenhuma. Manual, por quem desenvolve: `docker compose up -d` (Orion, Postgres), `uvicorn` e `ng serve`.	Não se aplica	Não
Dev	`develop` (único ambiente por branch): todo merge de PR.	Build, migrações, deploy e teste de fumaça	Q.A (1° ciclo), registrada no PR
Homologação (ADR-0018)	Tag `vX.Y.Z-rc.N` publicada na `release/X.Y.Z` pelo Tech Lead.	Build único da imagem, migrações, deploy, teste de fumaça e 3 cenários DKRT	Q.A (2° ciclo) e Tech Lead
Produção (ADR-0017)	Tag `vX.Y.Z` publicada na `main` pelo Tech Lead, após o merge da release.	Deploy do mesmo artefato após a aprovação	**Sim**, manual: Tech Lead
Self-hosted (ADR-0019)	Publicação da release.	Imagens versionadas publicadas no registry (GHCR)	Não. O produtor atualiza com `docker compose pull && up -d`

4.2 O que é automático e o que é manual

Automático	Manual (com responsável)
Build e testes Publicação da imagem e do build do frontend Migrações Alembic em Dev e homologação Deploy em Dev (push na `develop`) e em homologação (tag `-rc`) Teste de fumaça (`/health`) e de aceitação (3 cenários) Publicação do frontend (S3 e invalidação do CloudFront) e deploy das funções Lambda em produção, após a aprovação Verificação de saúde após o deploy	**Testes do Q.A** nos ciclos Dev e homologação **Criar a `release/X.Y.Z` e publicar a tag `-rc.N`** (Tech Lead) **Merge na `main` e publicação da tag `vX.Y.Z`** (Tech Lead) **Aprovar a implantação em produção** (Tech Lead ou substituto) **`terraform apply`** (disparo manual, em ambiente com aprovação: `workflow_dispatch`) Publicação de release self-hosted para clientes

4.3 Quem autoriza uma implantação em produção

O ambiente `producao` do GitHub Environments exige a aprovação do **Tech Lead** (ou do substituto designado pelo grupo). A opção *Prevent self-review* impede que quem disparou a release aprove a própria implantação quando houver outro autorizador disponível. Nenhum outro papel consegue promover um artefato para produção, e o Q.A não faz deploy: seu papel é aprovar ou reprovar a versão.

4.4 Banco de dados, rollback e infraestrutura

- **Migrações:** o `alembic upgrade head` roda antes de o novo código receber tráfego. Como código antigo e novo podem coexistir por instantes, as migrações seguem a regra *expandir e contrair*: primeiro adicionam (colunas anuláveis, tabelas novas), e só uma versão depois removem o que ficou sem uso. Isso evita depender de *downgrade*.
- **Mais de uma branch com migração:** duas branches criadas da mesma revisão geram duas *heads* no Alembic. O CI falha nesse caso (`alembic heads`); a segunda branch a ser liberada traz a `main` para dentro de si e reaponta sua migração antes da homologação. O banco do Dev, por misturar alterações ainda não aprovadas, pode ser recriado quando a `develop` for realinhada com a `main`.
- **Rollback:** reimplantar a tag anterior, já que os artefatos são imutáveis (versão e alias das Lambdas, build anterior do frontend com nova invalidação do CloudFront). Como as migrações são compatíveis, voltar o código não exige voltar o banco.
- **Infraestrutura (Terraform):** `plan` automático no PR, `apply` sempre manual. A infraestrutura muda raramente e um `apply` errado derruba o ambiente inteiro, então a aprovação humana vale o atrito.

> **Estado atual no repositório (verificado em 05/10/2026)**
>
> • A infraestrutura de produção (Lambda, ALB, CloudFront/S3, Parameter Store, VPC endpoints) e de homologação (EC2) está descrita em Terraform, em `environments/production` e `environments/homolog`. **Não há ambiente Dev em Terraform**: onde ele roda ainda precisa ser decidido (seção 9).
>
> • **Não há automação versionada** de build e push da imagem, nem de publicação do frontend ou atualização das Lambdas: hoje o repositório só automatiza Terraform. Esta é a principal lacuna de CD.
>
> • Pela ADR-0018 (15/09/2026), a conta AWS de homologação ainda não existia. O Terraform já está pronto para quando ela existir.
>
> • O ambiente de GitHub com aprovação manual já é usado no `terraform apply` (`academy-apply`, a ser renomeado para `homolog-apply`).

5. Tratamento de bugs em produção

5.1 Quando usar o atalho de hotfix

O **atalho** é para falha que afeta a produção e **não pode esperar** o ciclo completo: sistema indisponível, auditoria retornando resultado errado, falha de segurança. A issue recebe o rótulo `urgente`, e o Tech Lead confirma o uso do atalho. Qualquer outro bug segue o fluxo completo da seção 2, com uma branch `bug/`.

5.2 Passo a passo

1. **Registrar:** abrir uma issue `urgente` com a severidade, o impacto e como reproduzir, e avisar o grupo e o Tech Lead.
2. **Criar a branch a partir da `main`:** `git switch -c bug/<descricao>`. Como a `main` é a produção, a correção parte exatamente do que está em uso, e nada da `develop` vai junto.
3. **Corrigir com um teste que reproduz o bug**, escrito antes da correção. Mudança mínima, sem aproveitar para refatorar. Teste local completo.
4. **PR para a `main`** e envio da URL ao Tech Lead, com o CI completo. A revisão é obrigatória mesmo na urgência.
5. **Release de correção:** o Tech Lead cria `release/X.Y.Z` com o PATCH incrementado (de `1.2.0` para `1.2.1`), publica a tag `v1.2.1-rc.1` e a homologação roda o teste de fumaça, os 3 cenários do `dataspace-demo` e o caso que reproduz o bug. O Q.A aprova em um único ciclo, o da homologação.
6. **Publicar:** o Tech Lead faz o merge na `main`, publica a tag `vX.Y.Z` e aprova o deploy em produção.
7. **Devolver à `develop`:** a `main` é mergeada na `develop`, para que a correção não se perca no próximo ciclo e o Dev volte a ser igual à produção. Branches em andamento recebem a correção quando trouxerem a `main` para dentro delas (obrigatório antes do PR para a `main`).
8. **Encerrar:** apagar as branches, registrar na issue a causa raiz e o teste de regressão adicionado.

5.3 Se a correção demorar

Se a correção não for rápida, a primeira ação é **voltar a produção para a tag anterior** (seção 4.4), que restaura o serviço enquanto a branch `bug/` é preparada. O atalho elimina a etapa `develop`, mas não elimina nem a revisão do Tech Lead nem o Q.A na homologação.

> **Estado atual no repositório (verificado em 05/10/2026)**
>
> • Ainda não há tags nem pipeline de produção por tag, portanto o fluxo de correção descrito aqui é a meta. Até lá, uma correção emergencial seguiria o fluxo normal de PR com prioridade máxima.

6. Funcionalidades ainda não concluídas

6.1 Decisão: a branch de trabalho é a unidade de entrega

Neste fluxo, uma funcionalidade só chega à produção quando a **release da sua branch** é aprovada. Enquanto está incompleta, ela vive na branch de trabalho e, se for o caso, na `develop` (para o Q.A testar), e **nunca chega à `main`**. Em outras palavras, a branch faz o papel de isolamento que, em outros processos, seria feito por uma feature flag. A **feature flag fica como exceção** para o que precisa entrar em produção em partes.

6.2 Justificativa

- **Coerência com o fluxo.** Como as branches nascem da `main` e a release sai da própria branch, uma funcionalidade pela metade não contamina a produção nem as demais releases. Isso é exatamente o que o desenho em funil protege.
- **O Q.A testa a funcionalidade inteira**, no Dev e na homologação. Uma flag dividiria o teste entre os estados ligado e desligado, o que dobra o trabalho de uma equipe pequena.
- **Custo proporcional.** Variáveis em `Settings` já existem (`DEPLOYMENT_MODE`, `QUEUE_BACKEND`) e o endpoint `/session` já entrega configuração em runtime ao frontend (ADR-0020). Por isso a flag continua disponível sem custo extra quando for necessária.
- **Risco conhecido e tratado.** Branches longas geram conflitos: no próprio projeto, a branch `improve/refatoracao_simulador_data_space` viveu de 22/09 a 05/10 e exigiu resolução de conflitos no PR #19. As mitigações são a regra de 7 dias para branch parada (seção 1.4), a obrigação de trazer a `main` para dentro da branch antes da release, a divisão de trabalhos grandes em entregas menores e o isolamento de conflitos com a `develop` na `_develop`.

6.3 Branch ou flag?

Situação	Estratégia
Funcionalidade que cabe em uma ou duas semanas e será entregue de uma vez	**Branch.** Segue o fluxo normal e só vira release quando aprovada.
Trabalho grande que pode ser dividido em partes independentes	**Várias branches**, cada uma com sua release, sem expor nada ao usuário até a última.
Parte que precisa entrar em produção antes do todo (ex.: migração *expandir*) ou funcionalidade a ser liberada em momento diferente do deploy	**Feature flag** `FEATURE_<NOME>`, desligada por padrão, ligada por ambiente em `Settings`/Parameter Store e exposta ao frontend em `/session`. Removida em até 2 releases depois de ligada.
Experimento descartável (*spike*)	**Branch própria** com vida de até 5 dias, apagada ao final, nunca mergeada.

> **Estado atual no repositório (verificado em 05/10/2026)**
>
> • As flags de configuração existem e funcionam (`DEPLOYMENT_MODE`, `QUEUE_BACKEND` em `Settings`; `/session` já entrega `deployment_mode` e `tenant`). A convenção `FEATURE_*` e o campo `features` no `/session` são a formalização proposta, ainda não implementados.

7. Arquitetura da solução e pipelines

7.1 Mesma estratégia para tudo?

Sim. O projeto é um **monorepo**: backend, frontend, infraestrutura, bundle self-hosted e demonstração didática vivem no mesmo repositório, com uma só `main`, uma só `develop`, o mesmo fluxo de branches, o mesmo PR e o mesmo critério de aprovação. A arquitetura do código é um **monolito modular** (hexagonal), não microsserviços: há um único bounded context (auditoria de conformidade), um único banco e um único pacote Python que sobe como API ou como worker conforme o ponto de entrada.

Justificativa: numa equipe pequena, cada regra extra é custo de coordenação, e o contrato entre API e frontend muda junto (os DTOs do frontend espelham `schemas.py`), o que é mais simples de revisar e testar num mesmo PR, Dev e release.

7.2 Pipelines independentes por pasta

Mesma estratégia de branches, mas **pipelines independentes**, acionados por filtro de caminho (`paths:`), técnica que o workflow de infraestrutura já usa. Assim cada mudança só executa o que lhe diz respeito. A **versão**, porém, é uma só: a release `X.Y.Z` cobre API e frontend, que são liberados juntos.

Componente	Pipeline	Dispara com mudança em	Artefato	Implantável isoladamente?
API + Worker	`backend`	`src/**`, `migrations/**`, `tests/**`, `requirements.txt`, `Dockerfile`	Imagem de contêiner (ECR)	Sim. As três funções (api, worker, worker-dlq) sobem juntas, por design, pois usam a mesma imagem (ADR-0017)
Frontend	`frontend`	`frontend/**`	Build estático (S3/CloudFront)	Sim, desde que o contrato da API seja compatível
Infraestrutura	`infra`	`infra/terraform/**`	Plano Terraform	Sim, com `apply` manual
Bundle self-hosted	`selfhosted`	`selfhosted/**` e as de backend/frontend	Imagens no GHCR	Sim, publicado só em release
dataspace-demo	`demo`	`dataspace-demo/**`	Nenhum (não é implantado)	Não se aplica. O CI roda `test_seed_dkrt.py`; seus cenários servem de massa de teste de aceitação

7.3 Versionamento e componentes implantados separadamente

- **Uma versão de produto** (`vX.Y.Z`) cobre API e frontend. A infraestrutura é rastreada por commit e `plan`, sem tag própria. O `dataspace-demo` é versionado junto, por ser didático.
- **Contrato API e frontend:** mudança incompatível é feita em duas etapas (expandir, depois contrair) ou vira versão MAJOR. Com isso o frontend pode ser publicado sem a API e vice-versa.
- **SLM (Ollama):** não tem build próprio; usa a imagem pública e o modelo definidos no Terraform. Trocar o modelo é uma mudança de infraestrutura, aprovada como tal, e os testes *golden* detectam regressão de qualidade.

7.4 Quando isso deveria mudar

A decisão por monorepo e monolito modular não é definitiva. O grupo reavalia a separação em repositórios ou serviços quando **dois componentes passarem a precisar de ciclos de entrega diferentes na prática**. O

candidato mais provável é o worker de auditoria (`interfaces/worker`, função Lambda própria); a arquitetura hexagonal mantém essa extração barata.

> **Estado atual no repositório (verificado em 05/10/2026)**
>
> • O repositório já é um monorepo com `src/`, `frontend/`, `infra/`, `selfhosted/` e `dataspace-demo/`, e o workflow existente já usa filtro de caminho para a infraestrutura. Os demais pipelines por pasta ainda não existem.

8. Decisões e justificativas

Resumo das principais escolhas, com a alternativa descartada e o preço aceito. O enunciado pede coerência entre arquitetura, time, branches, integração e entrega: cada linha abaixo se apoia nas premissas da seção de contexto (equipe pequena, um produto, uma linha de produção, Q.A e Tech Lead como papéis).

Decisão	Alternativa descartada	Por que	Preço aceito
Fluxo em funil: `main` = produção, `develop` = Dev, `release/*` = homologação	GitHub Flow (só `main` e branches curtas com tags)	O time tem Q.A dedicado e quer dois ciclos de teste: um ambiente de integração antes da homologação reduz o que chega à release	Mais etapas, mais merges e mais ambientes para manter
Branches de trabalho nascem da `main`	Nascer da `develop` (Git Flow clássico)	A `develop` acumula trabalho ainda não aprovado; partindo da produção, a release leva só o que foi aprovado	Conflitos com a `develop` precisam ser resolvidos a cada alteração
Branch irmã `_develop` para chegar à `develop`	PR direto da branch de trabalho para a `develop`	Isola o merge da `develop` e seus conflitos, mantendo a branch de trabalho limpa para virar release	Uma branch e um passo manual a mais por alteração
Release nomeada (`release/X.Y.Z`) criada pelo Tech Lead; homologação e produção sobem por tag (`-rc.N` e `vX.Y.Z`)	Branch permanente `homologacao` ou deploy automático a cada push	Qualquer release pode ir a homologação e produção sem branch por ambiente. A tag é imutável e identifica exatamente o que foi testado; o deploy é uma decisão consciente do Tech Lead. Só o Dev, que muda o tempo todo, sobe por branch	Várias tags `-rc` e uma homologação por vez, que pode virar gargalo; o Tech Lead precisa publicar as tags
Merge na `main` só pelo Tech Lead	Qualquer integrante mergeia após aprovação	Um responsável pela entrega e menos risco de merge sem revisão ou sem aprovação do Q.A	Gargalo no Tech Lead; mitigado pelo substituto designado
Merge commit da release na `main`	Squash merge	A release preserva o histórico dos commits e fica visível no log; a tag aponta para um commit nomeado	Histórico da `main` menos linear
Build único, artefato promovido da homologação para produção	Recompilar a cada ambiente	O que foi testado na homologação é o que vai para produção	Exige release atualizada com a `main` antes da homologação
Hotfix com atalho (sem `develop`)	Bug urgente seguindo o fluxo completo	Urgência: remove só a etapa do Dev, mantendo revisão do Tech Lead e Q.A na homologação	O Dev recebe a correção só depois, por `main` → `develop`
Branch como unidade de entrega; flag como exceção	Flag para tudo, com a funcionalidade na `main` desligada	Coerente com o funil: o que não está aprovado não chega à `main`. O Q.A testa a funcionalidade inteira	Branches mais longas e risco de conflito (mitigado, seção 6.2)
Monorepo, pipelines por pasta	Multi-repositório ou microsserviços	Contrato API-frontend muda junto; um bounded context; equipe pequena	CI precisa de filtros de caminho bem mantidos
Terraform `apply` manual	Apply automático na `main`	Mudança rara e de alto impacto; hoje já é assim no workflow	Mais lento para infraestrutura

9. Estado atual e plano de adoção

Este manual descreve o processo-alvo. Para ser honesto sobre o ponto de partida, a tabela abaixo compara cada prática com o que **existe hoje no repositório** (verificado em 05/10/2026).

Prática	Hoje	Meta	Etapa
Branches e ambientes	Só `main` e branches de trabalho com nomes livres; sem ambiente Dev em Terraform (só `production` e `homolog`)	`develop`, `<branch>_develop` e `release/X.Y.Z`; decidir onde roda o Dev (por exemplo, docker compose em uma VM ou uma stack Terraform enxuta)	1
Proteção das branches	Não imposta (repositório privado em organização no plano gratuito)	Rulesets ativos em `main`, `release/*` e `develop`; merge na `main` restrito ao Tech Lead	1
PR, template e CODEOWNERS	PRs usados (#16 a #19); sem template nem CODEOWNERS	Templates (PR para `develop` e para `main`) com checklist; CODEOWNERS para `infra/`, `migrations/`, `docs/adr/`	1
CI de testes	Inexistente (só Terraform; `pip-audit` roda apenas em PRs que tocam `infra/`)	Pipelines `backend` e `frontend` bloqueando o merge, com `pip-audit`/`npm audit` em todo PR e checagem de uma só *head* do Alembic	1
Workflow Terraform	Aponta para `environments/academy` (aposentado, ADR-0018)	Instalar a versão `homolog` já preparada	1
Convenção de branches e commits	Nomes livres (`improve/*`); merges da `main` dentro de branches de trabalho	`feature/`, `improve/`, `bug/`; regras da seção 1.3; Conventional Commits	1
Qualidade de código	Sem configuração de `ruff` nem `mypy`	Adoção gradual com `ruff` e `mypy`	2
Tags e releases	Nenhuma tag; versão `0.1.0`	Release `1.0.0` na entrega; notas de release automáticas	2
CD do Dev e da homologação	Sem automação de build/push/deploy; conta AWS de homologação pendente (ADR-0018)	Deploy automático em `develop` (por branch) e na tag `vX.Y.Z-rc.N` (homologação), com teste de fumaça e aceitação	2
CD de produção	Apenas Terraform está automatizado	Tag `vX.Y.Z` na `main` com aprovação no Environment `producao`	3
Feature flags	Flags de configuração existem (`DEPLOYMENT_MODE`, `QUEUE_BACKEND`)	Convenção `FEATURE_*` e campo `features` em `/session`, para os casos de exceção	3
Hotfix e rollback	Sem fluxo formal	Atalho de hotfix e rollback por tag, ensaiados em homologação	3

Etapa 1 (fundação): criar a `develop` e a convenção de branches, proteger `main`, `release/*` e `develop`, criar templates e CODEOWNERS, ligar o CI de testes e corrigir o workflow de Terraform. **Etapa 2 (entrega):** qualidade de código, tags e deploy automático em Dev e homologação. **Etapa 3 (maturidade):** produção por tag com aprovação, feature flags formalizadas e ensaio de hotfix e rollback.

Referências

- Git Flow (Vincent Driessen): https://nvie.com/posts/a-successful-git-branching-model/
- GitHub Flow (alternativa considerada): https://docs.github.com/en/get-started/using-github/github-flow
- Conventional Commits: https://www.conventionalcommits.org
- Versionamento Semântico: https://semver.org/lang/pt-BR/
- Feature Toggles (Pete Hodgson): https://martinfowler.com/articles/feature-toggles.html
- HUMBLE, J.; FARLEY, D. Continuous Delivery. Addison-Wesley, 2010.
- Documentos do próprio projeto: ADR-0017 (produção em Lambda), ADR-0018 (homologação), ADR-0019 (modos SaaS e self-hosted), ADR-0020 (sessão e configuração em runtime), em `docs/adr/`.